

What the Kernel Sees: An Exploratory Study of Quantized LLM Inference on a 6 GB Consumer Laptop, and Pitfalls in Comparing Local Runtimes

Chuka Josemaria Uzo
Independent Researcher
ORCID: [0009-0009-9411-2858](https://orcid.org/0009-0009-9411-2858) chukauzo.j@gmail.com
DOI: [10.5281/zenodo.23074717](https://doi.org/10.5281/zenodo.23074717)

Abstract

Quantized language models in the GGUF format are routinely run on consumer hardware, and practitioners often choose between local runtimes on the basis of wall-clock comparisons. We report an exploratory measurement study of five quantized models (Mixtral-8x7B-Instruct at Q2_K; Llama-2-7B-Chat, Mistral-7B-Instruct, Zephyr-7B-alpha and Phi-2 at Q5_K_M) on a laptop with a 6 GB GPU, each run five times under two runtimes, WasmEdge and Python bindings to llama.cpp, with every run profiled by Linux `perf` counters. The data were collected for the author’s MSc thesis, which concluded that WasmEdge was 2–9× faster; here we re-analyse the original counters, inference logs and outputs without new experiments. The logs show that the comparison was not between runtimes. The Python build was CPU-only, although its log reported all layers as offloaded to the GPU, while the WasmEdge build used CUDA. Where both sides logged their phases, the GPU-enabled build processed prompts 17–20× faster but generated tokens only 1.4–1.7× faster, and prompt processing accounted for about 62% of the CPU-only run time for the 7B models. Kernel counters also exposed an instability that output-level evaluation would miss: under Python, Mixtral produced a byte-identical answer in all five runs, yet wall-clock time ranged from 2,371 s to 4,507 s, entirely in the generation phase, tracking context switches. Inspection of the outputs showed further confounds: the longest Python answers were degenerate repetition that ran until the context was full, and every WasmEdge answer began by fabricating a continuation of the article’s final quotation. We describe these confounds concretely and give a checklist for comparing local runtimes.

1 Introduction

Post-training quantization makes it possible to run instruction-tuned language models on hardware that a small organisation or an individual already owns (Detmers et al., 2022; Frantar et al., 2023; Lin et al., 2024). In practice this usually means a GGUF file run through llama.cpp (Gerganov et al., 2023) or a runtime built on it. Someone deploying such a model must choose a runtime, a quantization level, and settings such as how many layers to place on the GPU. These choices are often made from a handful of wall-clock timings.

Wall-clock time is a blunt instrument. It mixes model loading, prompt processing and token generation; it depends on how long the answer happens to be; and it is sensitive to everything else the operating system is doing. Computer-systems research has long warned that apparently simple performance comparisons are easy to get wrong (Mytkowicz et al., 2009; Georges et al., 2007; Kalibera and Jones, 2013), and inference benchmarks such as MLPerf fix many of these factors by design (Reddi et al., 2020). Informal comparisons of local LLM runtimes rarely do.

This paper reports an exploratory study that profiled every inference run of five quantized models with Linux `perf` counters (Gregg, n.d.) on a laptop with a 6 GB GPU. The data were collected for the author’s MSc thesis (Uzo, 2024), which compared WasmEdge (WasmEdge contributors, n.d.) with Python bindings to llama.cpp (Betlen et al., 2023) and reported that WasmEdge was 2–9× faster. Re-examining the original counters, the llama.cpp logs and the model outputs, we find that this conclusion does not hold: the two

conditions differed in GPU support, and their outputs differed in ways that wall-clock time cannot reveal. We present the study for what it can support and use its shortcomings as a worked example.

Our contributions are:

1. A per-run dataset of `perf` counters, llama.cpp phase timings, GPU telemetry and outputs for five quantized models under two runtimes on a 6 GB consumer GPU laptop, re-analysed from the original logs (Section 3.6).
2. A phase-level account showing that the reported runtime gap is explained by GPU-accelerated prompt processing in one build and its absence in the other, which a misleading log line had concealed (Sections 4.1 and 4.2).
3. A case in which kernel counters revealed a nearly two-fold run-to-run slowdown for byte-identical outputs, confined to the generation phase (Section 4.4).
4. Evidence that output length, the usual normaliser for such comparisons, was itself contaminated by degenerate repetition and prompt-formatting artefacts (Section 4.5), and a checklist for avoiding these problems (Section 5).

2 Background and related work

Quantized models on consumer hardware. Post-training quantization reduces weights to fewer bits to cut memory use and speed up inference (Detrmers et al., 2022; Frantar et al., 2023; Lin et al., 2024). llama.cpp and its GGUF format provide block-wise formats such as Q2_K and Q5_K_M and can split a model between CPU and GPU by offloading a chosen number of layers (Gerganov et al., 2023). Inference has two phases with different costs: prompt processing, which handles the input tokens in batches and is dominated by large matrix multiplications, and generation, which produces one token at a time and is largely limited by memory bandwidth. Effects of quantization on model behaviour, rather than speed, have been studied for larger models (Hong et al., 2024) and, for small models and GGUF formats, in our companion work (Uzo, 2026). The present paper does not vary quantization within a model and makes no claims about the effect of quantization level.

Runtimes. WasmEdge is a WebAssembly runtime whose LLM support, at the time of the study, executed GGUF models through a llama.cpp-based backend (WasmEdge contributors, n.d.). The Python condition used llama.cpp-python, which binds the same C/C++ library (Betlen et al., 2023). Both conditions therefore executed llama.cpp kernels; differences between them reflect build options, configuration and the host program rather than WebAssembly versus Python as languages.

Benchmarking methodology. Mytkowicz et al. (2009) showed that innocuous setup differences can bias performance measurements; Georges et al. (2007) and Kalibera and Jones (2013) argued for repeated runs, reporting of variation and separation of warm-up from steady state. MLPerf Inference fixes models, data, accuracy targets and scenarios so that systems can be compared (Reddi et al., 2020). Our study met few of these standards, which is part of what it can teach.

Fabrication and caution. Language models can produce fluent but unsupported content (Ji et al., 2023), and have some ability to recognise what they do not know (Kadavath et al., 2022). Section 4.5 touches on this behaviour.

3 Study design

3.1 Hardware and software

All runs used an Acer Predator G3-571 laptop with an Intel Core i7-7700HQ CPU (4 cores, 8 threads, 2.8 GHz base), 32 GB of DDR4 RAM, an NVIDIA GeForce GTX 1060 Mobile GPU (GP106M, compute capability

6.1) with 6 GB of VRAM, CUDA 12.3, and Ubuntu 22.04 LTS. Software versions were WasmEdge 0.13.5, Python 3.10.12 and `perf` 6.5.8. The version of `llama-cpp-python` was not recorded. All runs took place on 28 February 2024.

3.2 Models

Five instruction- or chat-tuned models were downloaded as pre-quantized GGUF files (Table 1). Models were chosen to fit the hardware, and the quantization level differs between Mixtral and the others; quantization level is therefore confounded with model and is not an object of study here.

Table 1: Models. File sizes are as listed by the GGUF publishers. Mixtral is a sparse mixture-of-experts model with about 12.9B parameters active per token. Zephyr-7B-alpha is a fine-tune of Mistral-7B and shares its architecture.

Model	Parameters	Format	File size (GB)
Mixtral-8x7B-Instruct-v0.1 (Jiang et al., 2024)	46.7B	Q2_K	15.64
Llama-2-7B-Chat (Touvron et al., 2023)	7B	Q5_K_M	4.78
Mistral-7B-Instruct-v0.1 (Jiang et al., 2023)	7B	Q5_K_M	5.13
Zephyr-7B-alpha (Tunstall et al., 2023)	7B	Q5_K_M	5.10
Phi-2 (Jawaheripi and Bubeck, 2023)	2.7B	Q5_K_M	2.07

3.3 Task and input

Every run gave the model the same input: a single English news article of about 1,050 words on freight-rate movements following disruption to Red Sea shipping (LaRocco, 2024), together with a fixed instruction asking for a concise structured summary with ten fields (entities involved, policy designations, entity profile, competitors, market valuation, impact on stocks, industries affected, tone, a risk rating from 0 to 10, and a justification). Several fields, notably market valuation, competitors and impact on stocks, could not be answered from the article. The prompt was passed as plain text without a model-specific chat template; tokenized, it was 1,367–1,563 tokens long depending on the model’s tokenizer and the runtime’s prompt layout. The domain of the article is incidental to this study; it serves as a fixed, realistic long input.

3.4 Runtime configurations

Table 2 lists the settings passed to each runtime. They were not matched between runtimes and, under WasmEdge, were adjusted per model to avoid crashes. As Section 4.1 shows, several of these settings did not take effect as written. Decoding parameters were not recorded; Python runs produced byte-identical outputs across repetitions, consistent with a fixed random seed, while WasmEdge outputs varied between runs.

Table 2: Settings passed to each runtime (WasmEdge / Python). Section 4.1 reports what the logs show actually ran. Zephyr produced no output under Python in any run.

Model	Context (tokens)	Max new tokens	GPU layers	Batch
Mixtral-8x7B	10,000 / 8,024	10,000 / 10,000	5 / 100	624 / 3,024
Llama-2-7B	8,024 / 8,024	10,000 / 10,000	10 / 100	3,024 / 3,024
Mistral-7B	524 / 8,024	600 / 10,000	5 / 100	524 / 3,024
Zephyr-7B	8,024 / –	10,000 / –	5 / –	524 / –
Phi-2	524 / 8,024	600 / 10,000	10 / 100	524 / 3,024

3.5 Procedure and measurements

Each model was run five times under each runtime. Each run was wrapped in `perf stat`, which recorded wall-clock, user and system time, task clock, context switches, CPU migrations, page faults, cycles, instructions,

branches and branch misses. These are counts of CPU-side events: work executed on the GPU does not appear in them. The combined standard output and error of each run were saved, which preserved the model’s answer and, where logging was enabled, llama.cpp’s load-time messages and phase timings (prompt-processing and generation time and token counts). Phase timings are available for all Python runs and for the WasmEdge runs of Phi-2 and Zephyr. For the WasmEdge runs of Mixtral, Llama-2 and Mistral, WasmEdge’s detailed inference logging was switched off (`Log enable: false`), so their logs contain the run settings and the answer but not llama.cpp’s load messages or phase timings. Under WasmEdge only, `nvidia-smi` and `free` were sampled every second to log GPU utilisation, VRAM, power draw and system memory. We report the ratio of user time to wall-clock time as “effective cores”, the average number of CPU threads kept busy.

The author rated every output on ten five-point scales (for example relevance, accuracy, fluency, and a “tone of cautiousness” scale from 1, confident, to 5, cautious and non-committal). Ratings were made by a single rater who knew which model and runtime produced each output.

3.6 Data and analysis

We re-analysed the original `perf` outputs, run logs and GPU logs, and the measurement workbook whose per-run tables are reproduced in the thesis appendix (Uzo, 2024); every value in the workbook matched the raw files. We found one error in the workbook: the formulas producing the Python summaries averaged runs 1–4 only, although the results were labelled as averages of five runs. Here every average uses all five runs. Output repetition was measured as the share of word 8-grams in an answer that repeat an earlier 8-gram. With $n = 5$ per configuration, we report descriptive statistics and simple linear fits, give correlation coefficients as descriptions rather than tests, and draw no inferential conclusions.

4 Results

4.1 What actually ran

The logs contradict the configuration in three ways.

The Python build had no GPU support. Every Python log reports “offloaded 33/33 layers to GPU”, but the same logs show all model weights, the key-value cache and the compute buffers allocated in CPU memory (for Mixtral, a 14,919 MiB CPU weight buffer, far larger than the 6 GB GPU), no CUDA initialisation, and `BLAS = 0` in the system information line. The llama-cpp-python package had been built without GPU support, and the request for 100 GPU layers was silently ignored while the log reported it as fulfilled. The effective-core figures agree: Python runs kept 4.8–6.2 CPU threads busy on average.

The WasmEdge build used CUDA. Every WasmEdge log shows CUDA initialising on the GTX 1060, and the logged runs show 5 (Zephyr) and 10 (Phi-2) of 33 layers placed on the GPU. In a CUDA build, llama.cpp of this period could also use the GPU for the large matrix multiplications of prompt processing even for layers held in CPU memory, which is consistent with the phase timings below.

Context settings did not take effect as written. Where WasmEdge logged its context, it used the model’s trained context length rather than the requested value: 2,048 tokens for Phi-2 (requested 524) and 32,768 for Zephyr (requested 8,024). Both models processed the whole prompt (1,398 and 1,523 tokens). Mistral’s WasmEdge logs lack this line because detailed logging was switched off for those runs, but its answers refer to material from the end of the article, so it also processed the full input.

4.2 Where the time went

Table 3 decomposes the runs for which phase timings exist. Under the CPU-only Python build, prompt processing ran at 6.2–7.3 tokens/s for the 7B-class models and 14.5–14.8 tokens/s for Phi-2. For Llama-2 and Mistral, which produced short answers, prompt processing took about 220 s of roughly 355 s, around 62% of the run. Generation ran at 2.8–3.5 tokens/s for the 7B models and 5.3–6.0 tokens/s for Phi-2.

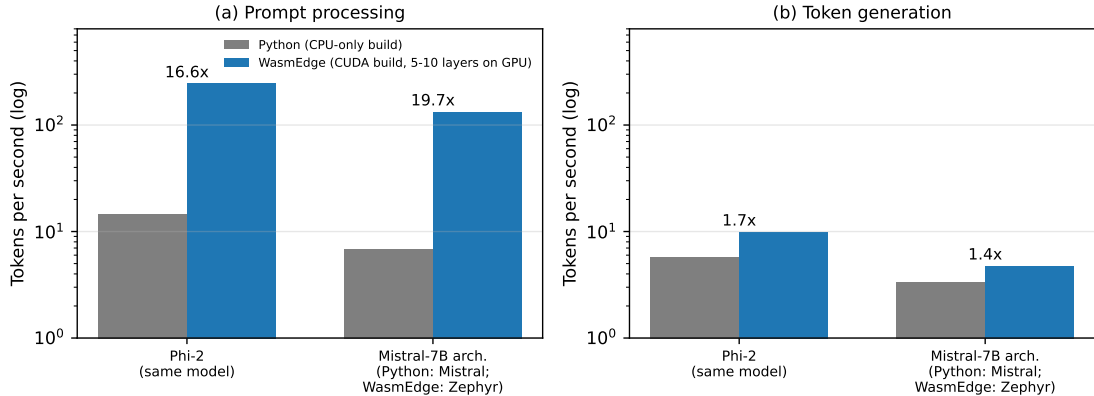


Figure 1: Prompt-processing and generation throughput (log scale) for the CPU-only Python build and the CUDA-enabled WasmEdge build. The second pair compares Python Mistral-7B with WasmEdge Zephyr-7B, which share architecture and quantization format but not weights.

The WasmEdge build was very much faster at prompt processing and only moderately faster at generation (Figure 1). For Phi-2, the only model with phase timings under both builds, prompt processing ran at 244 versus 14.7 tokens/s (16.6 $\times$) and generation at 9.9 versus 5.8 tokens/s (1.7 $\times$). Zephyr, which has no Python results, shares Mistral-7B’s architecture and quantization format, so its WasmEdge rates give an approximate comparison with Python Mistral: 133 versus 6.8 tokens/s for prompt processing (19.7 $\times$) and 4.8 versus 3.3 tokens/s for generation (1.4 $\times$). This is the profile expected from GPU-accelerated batched matrix multiplication with most layers still on the CPU. The “runtime” difference reported in the thesis is therefore best explained by a GPU-enabled build compared against a CPU-only one, amplified by differences in output length, and not by WebAssembly or Python.

Table 3: Phase timings from llama.cpp logs, means of five runs (ranges in brackets). “Prompt share” is prompt-processing time as a share of the logged total. WasmEdge phase timings were logged only for Phi-2 and Zephyr.

Build	Model	Prompt tok.	Prompt tok/s	Gen. tok.	Gen. tok/s	Prompt share
Python (CPU)	Mixtral-8x7B	1,493	6.5 [6.2–6.9]	6,530	2.6 [1.6–3.4]	8%
	Llama-2-7B	1,563	7.2 [7.0–7.3]	445	3.2 [2.8–3.5]	61%
	Mistral-7B	1,493	6.8 [6.7–6.8]	441	3.3 [3.3–3.4]	62%
	Phi-2	1,367	14.7 [14.5–14.8]	2,500	5.8 [5.3–6.0]	17%
WasmEdge (CUDA)	Phi-2	1,398	244 [237–248]	599	9.9 [9.7–10.2]	9%
	Zephyr-7B	1,523	133 [131–134]	78–587	4.8 [4.6–4.9]	9–42%

The logs also separate model loading and process start-up from inference. The first Python run of each model spent 18–45 s outside inference (Llama-2 18 s, Phi-2 31 s, Mistral 45 s) against 2–4 s in later runs, a cold-start cost from reading the model file. Two Zephyr runs under WasmEdge show the same effect (about 72 s outside inference against about 10 s in the other three).

4.3 Within a configuration, output length tracks wall-clock time

For the WasmEdge runs of Mixtral, Llama-2 and Mistral, which have no phase timings, answer length varied from run to run while model, input and settings were fixed. Wall-clock time rose close to linearly with words generated (Figure 2a; $r = 1.00, 0.89$ and 0.94 ; slopes 0.36, 0.28 and 0.25 s per word; intercepts 30–40 s). The intercepts are larger than the logged prompt-processing time of the comparable Zephyr runs (about 11 s), which suggests they also absorb model loading and, in some runs, cold starts; with five runs per model the fits are descriptive only. The practical implication is the same as in Section 4.2: comparisons that do not fix the number of generated tokens partly measure answer length rather than speed.

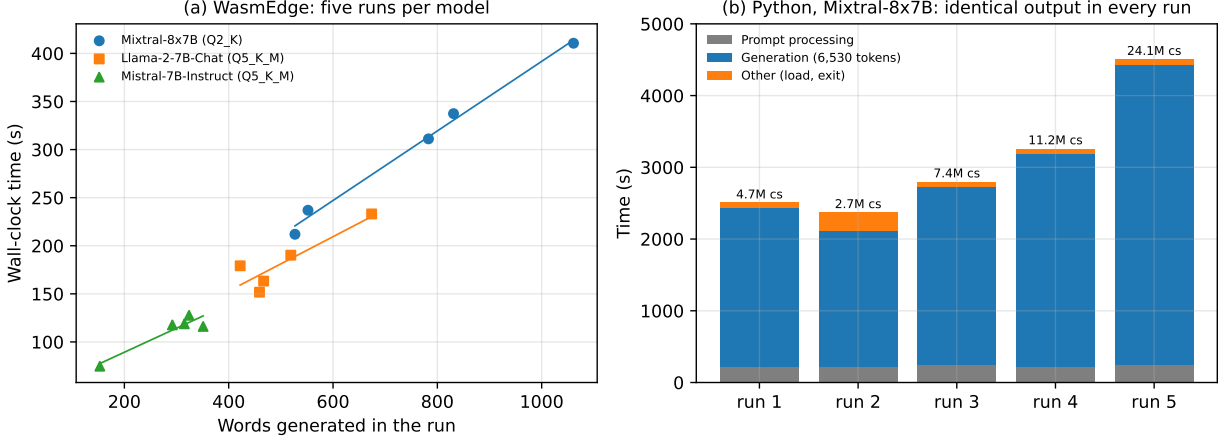


Figure 2: (a) WasmEdge: wall-clock time against words generated, with a least-squares line per model. (b) Python, Mixtral-8x7B: time per run split into prompt processing, generation and other time, with context switches above each bar. The output was byte-identical in all five runs.

4.4 Identical outputs, unstable cost

Under Python, every model produced a byte-identical answer in all five runs, so any variation in cost comes from the system, not from the model’s work. For Llama-2, Mistral and Phi-2 this variation was small: wall-clock time varied by 3.7–5.6% (coefficient of variation), mostly from the first-run cold start. For Mixtral it was not (Figure 2b). Wall-clock time ranged from 2,371s to 4,507s, a factor of 1.9, and rose steadily from run 2 to run 5. The phase timings place the slowdown entirely in generation: prompt processing took 216–242s in every run, while generation of the same 6,530 tokens slowed from 3.4 to 1.6 tokens/s. Over the same runs, context switches rose from 2.7 to 24.1 million and system time from 626s to 2,903s; across the five runs, wall-clock time correlated with context switches at $r = 1.00$ and with system time at $r = 0.99$. Page faults were nearly constant (about 504,000 per run).

The counters do not identify the cause. Mixtral at Q2_K holds about 15.6 GB (14,919 MiB) of weights, and the run used an 8,024-token cache on a machine with 32 GB of RAM, so memory pressure from other processes, thermal throttling during back-to-back runs and contention among inference threads are all plausible. What the counters show is that the slowdown was a system-level phenomenon in the memory-bound generation phase: the model did the same work each time, but the operating system spent increasingly more effort scheduling it. An evaluation that recorded only outputs, or only a mean time, would have missed this.

4.5 What the outputs reveal

Reading the outputs exposed two further confounds.

The longest answers were degenerate. Python Mixtral generated 6,530 tokens, which with its 1,493-token prompt filled the 8,024-token context exactly: generation stopped only because the context ran out. After a reasonable opening, the answer repeated the same passages, and 90% of its word 8-grams were repeats. Python Phi-2 began by echoing the instruction and the article and then produced fragmentary text until it stopped at 2,500 tokens. The thesis noted “defective duplicates” in Python output; the logs show that these drove the large word counts used in its comparisons. Per-word normalization of time therefore compares a fluent answer on one side with a loop on the other. Repetition was not confined to Python: some WasmEdge Mixtral answers also repeated substantial passages (up to 46% of 8-grams), which inflates the word counts used in Section 4.3.

Every WasmEdge answer began with a fabricated quotation. The prompt placed the article after the instruction, and the article as supplied ended in the middle of a quotation from a named executive,

Table 4: Output checks. “Repetition” is the share of word 8-grams that repeat an earlier 8-gram in the same answer (Python answers were identical across runs; WasmEdge values are ranges over five runs). “Fabricated opening” counts answers that began with quoted speech not present in the article.

Build	Model	Repetition	Fabricated opening	How generation ended
Python (CPU)	Mixtral-8x7B	0.90	0/5	Context full (8,023 of 8,024 tokens)
	Llama-2-7B	0.00	0/5	Model stopped (445 tokens)
	Mistral-7B	0.00	0/5	Model stopped (441 tokens)
	Phi-2	0.07	0/5	Echoed prompt, then fragments (2,500 tokens)
WasmEdge (CUDA)	Mixtral-8x7B	0.08–0.46	5/5	Model stopped
	Llama-2-7B	0.00–0.15	5/5	Model stopped
	Mistral-7B	0.00–0.21	5/5	Model stopped
	Zephyr-7B	0.00–0.04	5/5	Model stopped (78–587 tokens)
	Phi-2	0.00–0.14	5/5	Token limit (599 of 600)

without its closing quotation mark. With no chat template, every WasmEdge answer, for all five models and all runs, began with fabricated quoted speech, in almost every case a continuation of that quotation that the person never said, before turning to the requested summary. In one Zephyr run the answer consisted only of such a quotation followed by a fabricated statistic attributed to a company named in the article. In one Phi-2 run the model went on to write itself a new, unrelated task and complete it. Apart from the fabricated opening, the WasmEdge answers of Mixtral, Llama-2 and Mistral were generally coherent structured summaries, consistent with the higher quality ratings they received in the thesis, although they still contained unsupported specifics (for example, figures on the number of data points and carriers covered by a benchmarking firm that the article did not give). Python answers did not show this behaviour; the Python script’s exact prompt layout was not preserved, so we cannot say why. Prompt format thus changed both the length and the factual content of the outputs.

An anecdotal observation on declining versus fabricating. Because the prompt asked for fields the article could not support, the outputs also showed how each model handled unanswerable requests. Mixtral and Mistral under WasmEdge typically stated that no market valuation or stock impact was mentioned, and Llama-2 did so under Python. Zephyr and Phi-2 under WasmEdge more often supplied specific, unsupported content, such as a trade-growth forecast attributed to an international organisation that the article did not mention. On the cautiousness scale, WasmEdge Mixtral and Mistral averaged 5.0, Llama-2 3.2, Zephyr 1.8 and Phi-2 1.0. This observation is anecdotal. It rests on one article and one rater who was not blind to condition, it was not stable across builds (Mixtral’s looping Python answer was rated 1), and it sits alongside the fact that even the “cautious” WasmEdge answers opened with a fabricated quotation.

4.6 GPU and memory telemetry

Under WasmEdge, GPU memory rose to a plateau when the model was loaded and stayed there until the run ended (Figure 3a). The plateau reflected the layers placed on the GPU plus buffers: about 5,880 MiB (96% of the 6 GB) for Mixtral, 4,270 MiB (69%) for Llama-2, 3,820–3,860 MiB (62–63%) for Mistral and Zephyr, and 1,230 MiB (20%) for Phi-2. In the first Zephyr run, GPU memory was not allocated until about 60 s after logging began, the cold start seen in the timing logs (Section 4.2). Mean GPU utilisation over all runs was low, 11–16%. Power draw peaked at 84–113 W during model loading and prompt processing and otherwise fluctuated around 25–40 W during generation (Figure 3b). Memory reported as used by `free` stayed at about 3–4 GB; because llama.cpp memory-maps model files, the weights appear as page cache rather than used memory, so this figure understates the models’ footprint.

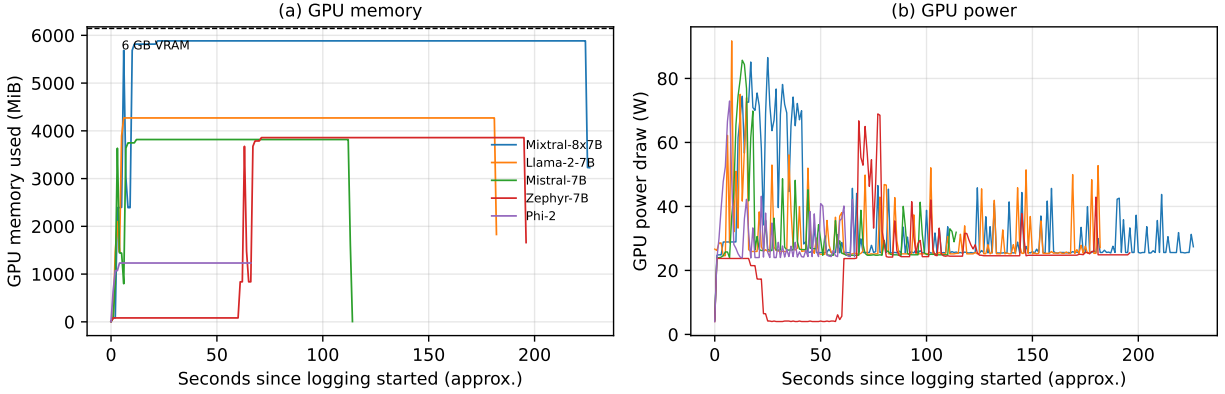


Figure 3: GPU telemetry for the first WasmEdge run of each model, sampled about once per second: (a) GPU memory in use, (b) power draw. Zephyr’s delayed rise in memory reflects a cold start.

5 Discussion: a checklist for comparing local runtimes

The original study set out to compare runtimes and ended up demonstrating how such a comparison goes wrong. Its central error was not in the measurements, which were accurate, but in assuming that the configuration described what ran. The following practices would have made the comparison attributable.

1. **Verify what ran, not what was requested.** Check the backend (for llama.cpp, CUDA initialisation and where buffers are allocated), the actual context size and the number of layers on each device. A log line saying layers were offloaded is not evidence that they were.
2. **Hold settings constant** across the conditions being compared: build options, GPU layers, context size, batch size, thread count and generation limit.
3. **Fix the work.** Use the same prompt layout, chat template and decoding parameters, or force a fixed number of generated tokens, so that both conditions produce comparable output.
4. **Report prompt processing and generation separately**, as tokens per second, rather than total time per run.
5. **Read the outputs.** Check for repetition, echoed prompts and runaway generation before using output length or quality in any comparison.
6. **Instrument every condition the same way**, including GPU telemetry, and remember that CPU counters do not capture GPU work.
7. **Report every run**, separate cold starts from warm runs, and check that summary statistics use all runs.
8. **Keep the logs.** The confounds reported here were recoverable only because the original logs and outputs had been saved.

Within these limits, the study supports one positive point: kernel counters and inference logs are a cheap and useful addition to local LLM evaluation. They require no change to the model or runtime, and in this study they exposed a near two-fold slowdown that output-level evaluation would have reported as identical runs, as well as the build difference that explained the headline result.

6 Limitations

This is an exploratory re-analysis of data from a single laptop, five models, one input and five runs per condition. Phase timings exist for all Python runs but only for the WasmEdge runs of Phi-2 and Zephyr, and the Mistral-architecture comparison pairs different fine-tunes. No GPU telemetry exists for Python runs, although the logs establish that they did not use the GPU. Decoding parameters and the llama-cpp-python version were not recorded. Each model was tested at one quantization level, and quantization level is confounded with model. Quality ratings come from one rater who was not blind to condition. All correlations are based on five points and are descriptive. The study used model and software versions available in early 2024, and both runtimes have changed considerably since.

7 Conclusion

We profiled five quantized GGUF models on a 6 GB consumer GPU laptop under two runtimes, with kernel counters, inference logs and outputs recorded for every run. The apparent speed advantage of one runtime over the other was, on inspection, the advantage of a GPU-enabled build over a CPU-only build, whose log nonetheless reported full GPU offload; the difference lay almost entirely in prompt processing. Output length, the obvious normaliser, was contaminated by degenerate repetition on one side and fabricated quotations on the other. Kernel counters revealed a near two-fold slowdown in the generation phase for byte-identical outputs. For practitioners, the lesson is that a fair comparison of local runtimes requires verifying what actually ran, fixing the work, and reading the outputs, and that keeping counters and logs alongside the outputs is an inexpensive way to find out when a comparison has gone wrong.

Relationship to prior work by the author

This paper is derived from the author’s MSc thesis (Uzo, 2024), re-analysing its recorded measurements, logs and outputs without new experiments, and supersedes the thesis’s runtime-comparison conclusions. It is distinct from the author’s study of quantization and prompt-injection safety in small models (Uzo, 2026), which uses different models, data and experiments.

Data availability

The raw `perf` outputs and run logs, a per-run table of counters and phase timings, the GPU telemetry (raw and with corrected column labels) and the script that generates the figures are released with this preprint. Every passage of ten or more consecutive words from the copyrighted article has been removed from prompts and outputs.

Use of AI assistance

An AI assistant (Claude, Anthropic) was used to re-analyse the recorded measurements and logs, to identify confounds and inconsistencies in the original analysis, and to draft this manuscript. The author designed and ran the original experiments, performed all quality ratings, checked the reported values against the original data, and takes full responsibility for the content.

References

- Andrei Betlen et al. llama-cpp-python: Python bindings for llama.cpp. <https://github.com/abetlen/llama-cpp-python>, 2023.
- Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. LLM.int8(): 8-bit matrix multiplication for transformers at scale. In *Advances in Neural Information Processing Systems*, 2022.

- Elias Frantar, Saleh Ashkboos, Torsten Hoefer, and Dan Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *International Conference on Learning Representations*, 2023.
- Andy Georges, Dries Buytaert, and Lieven Eeckhout. Statistically rigorous Java performance evaluation. In *Proceedings of the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages and Applications (OOPSLA)*, 2007.
- Georgi Gerganov et al. llama.cpp: LLM inference in C/C++. <https://github.com/ggml-org/llama.cpp>, 2023.
- Brendan Gregg. perf-tools: Performance analysis tools based on Linux perf_events and ftrace. <https://github.com/brendangregg/perf-tools>, n.d.
- Junyuan Hong et al. Decoding compressed trust: Scrutinizing the trustworthiness of efficient LLMs under compression. In *International Conference on Machine Learning*, 2024.
- Mojan Javaheripi and Sébastien Bubeck. Phi-2: The surprising power of small language models. Microsoft Research Blog, published 12 December, <https://www.microsoft.com/en-us/research/blog/phi-2-the-surprising-power-of-small-language-models/>, 2023.
- Ziwei Ji et al. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12), 2023.
- Albert Q. Jiang et al. Mistral 7B. *arXiv preprint arXiv:2310.06825*, 2023.
- Albert Q. Jiang et al. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.
- Saurav Kadavath et al. Language models (mostly) know what they know. *arXiv preprint arXiv:2207.05221*, 2022.
- Tomas Kalibera and Richard Jones. Rigorous benchmarking in reasonable time. In *Proceedings of the 2013 International Symposium on Memory Management (ISMM)*, 2013.
- Lori Ann LaRocco. Red Sea supply chain inflation may be peaking already, new trade data suggests. CNBC, published 31 January, 2024.
- Ji Lin et al. AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration. In *Proceedings of Machine Learning and Systems*, 2024.
- Todd Mytkowicz, Amer Diwan, Matthias Hauswirth, and Peter F. Sweeney. Producing wrong data without doing anything obviously wrong! In *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2009.
- Vijay Janapa Reddi et al. MLPerf inference benchmark. In *Proceedings of the 47th Annual International Symposium on Computer Architecture (ISCA)*, 2020.
- Hugo Touvron et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.
- Lewis Tunstall et al. Zephyr: Direct distillation of LM alignment. *arXiv preprint arXiv:2310.16944*, 2023.
- Chuka Josemaria Uzo. Performance analysis and comparison of quantized language models on resource constraints: The case of supply chain risk data. MSc thesis, IU International University of Applied Sciences. Zenodo, <https://doi.org/10.5281/zenodo.13829349>, 2024.
- Chuka Josemaria Uzo. Capable enough to be hijacked: Quantization, caution, and indirect prompt injection in small edge LLMs. Preprint, Zenodo, <https://doi.org/10.5281/zenodo.22965139>, 2026.
- WasmEdge contributors. WasmEdge: A lightweight, high-performance and extensible WebAssembly runtime. <https://github.com/WasmEdge/WasmEdge>, n.d.